

ARTIFICIAL INTELLIGENCE

Unit II — Lecture Notes

Search Strategies & Game Playing

Uninformed · Heuristic · Adversarial Search

JD College of Engineering & Management, Nagpur

Academic Year 2025–26

1. Introduction to Search in AI

Search is one of the most fundamental paradigms in Artificial Intelligence. When an AI agent needs to find a sequence of actions that takes it from an initial state to a goal state, it performs a search through the space of all possible states. This collection of states and transitions is called the state space.

DEFINITION

A search problem consists of five components:

- (1) Initial State — where the agent starts
- (2) Actions (Operators) — moves available at each state
- (3) Transition Model — result of applying each action
- (4) Goal Test — determines if current state is the goal
- (5) Path Cost — numeric cost assigned to each path

REAL-LIFE ANALOGY

Imagine you are lost in a city and need to find the nearest hospital. Your current location is the initial state. The hospital is the goal. Every road you can take is an action. The entire city map is the state space. Search algorithms decide which roads to explore first to reach the hospital efficiently.

Two Broad Categories of Search

Uninformed (Blind) Search: No additional knowledge about the problem beyond the definition. Explores states blindly.

Informed (Heuristic) Search: Uses domain-specific hints (heuristics) to guide exploration more intelligently toward the goal.

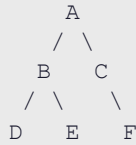
Adversarial Search (Game Playing): Deals with situations where an opponent is actively working against the agent's goals.

2. Breadth-First Search (BFS)

BFS explores the state space level by level. It starts at the root node, visits all nodes at depth 1, then all nodes at depth 2, and so on. It uses a Queue (FIFO) data structure to track which node to visit next. BFS is guaranteed to find the shallowest solution first.

WORKED EXAMPLE — FINDING SHORTEST PATH

Consider cities A, B, C, D, E, F connected in a tree. BFS from A will first visit all cities directly connected to A (depth 1: B, C), then all cities reachable in 2 hops (depth 2: D, E, F). The first time BFS reaches the goal, it has used the fewest steps possible.



BFS Order of Exploration:

```

Queue: [A]
Visit A --> Enqueue B, C      Queue: [B, C]
Visit B --> Enqueue D, E      Queue: [C, D, E]
Visit C --> Enqueue F         Queue: [D, E, F]
Visit D --> (no children)      Queue: [E, F]
Visit E --> GOAL FOUND! ✓

```

BFS visits: A -> B -> C -> D -> E (Level by Level)

Fig 1: BFS traversal – goal E found via shortest path A->B->E

Properties of BFS

Property	Value	Meaning
Complete?	Yes	Will always find a solution if one exists (finite space)
Optimal?	Yes (unit cost)	Finds the shallowest (fewest steps) solution
Time Complexity	$O(b^d)$	b = branching factor, d = depth of solution
Space Complexity	$O(b^d)$	Stores all nodes in memory — can be very large

INDUSTRY APPLICATIONS

Google Maps / GPS: Finding the shortest route with minimum number of hops (road segments).

LinkedIn / Facebook: 'People You May Know' — finds connections at 1-hop, 2-hop distance.

Web Crawlers: Google's crawler explores the web layer by layer, starting from seed URLs.

Puzzle Solving: Rubik's Cube solvers, 8-puzzle — finding minimum number of moves.

3. Depth-First Search (DFS)

DFS explores as deep as possible along one branch before backtracking and trying other branches. It uses a Stack (LIFO) — or recursion — to track the current path. Unlike BFS, DFS may find a solution quickly, but it might not be the shortest one.

WORKED EXAMPLE — MAZE SOLVING

Think of navigating a maze. DFS picks one corridor and walks as far as possible. If it hits a dead end, it backtracks to the last junction and tries another corridor. It will eventually find the exit — but not necessarily via the shortest path.



DFS Order (left-first strategy):

Stack: [A]

Visit A --> Push C, B Stack: [C, B]

Visit B --> Push E, D Stack: [C, E, D]

Visit D --> (no children) Stack: [C, E]

Visit E --> GOAL FOUND! ✓

DFS visits: A -> B -> D -> E (Deep dive down left branch)

Fig 2: DFS dives deep into one branch. Found E faster here, but this is not always the case.

BFS vs DFS — Comparison

Feature	BFS	DFS
Data Structure	Queue (FIFO)	Stack (LIFO)
Complete?	Yes	No (can loop in infinite spaces)
Optimal?	Yes (unit cost)	No
Memory Usage	High — all frontier nodes stored	Low — only current path stored
Best Used When	Shortest path is needed	Memory is limited; solutions are deep

INDUSTRY APPLICATIONS

Compilers: Parsing expression trees, detecting cycles in dependency graphs.

File Systems: Linux/Windows directory traversal — going deep into nested folders.

Network Analysis: Detecting connected components and cycles in networks.

Backtracking Problems: Sudoku solvers, N-Queens, crossword puzzle generators.

4. Search with Partial Information — Heuristic Search

Uninformed search is like exploring a building blindfolded. Heuristic search gives the agent an educated guess — a function $h(n)$ that estimates how far a node n is from the goal. This allows the agent to prioritize promising paths and avoid wasting time on dead-ends.

DEFINITION — HEURISTIC FUNCTION $H(N)$

A heuristic $h(n)$ estimates the cost from node n to the goal state.

A heuristic is called **ADMISSIBLE** if it never overestimates the true cost — it is always optimistic.

An admissible heuristic guarantees that algorithms like A* find the optimal solution.

Example: Straight-line distance (Euclidean) to goal is admissible for road maps — the actual road distance is always $\geq$ straight-line distance.

REAL-LIFE ANALOGY

When you drive to a destination and can see the building on the horizon, you naturally choose the road pointing toward it. The straight-line distance to that building is your heuristic — you don't know road blocks yet, but this gives a useful, optimistic estimate of remaining distance.

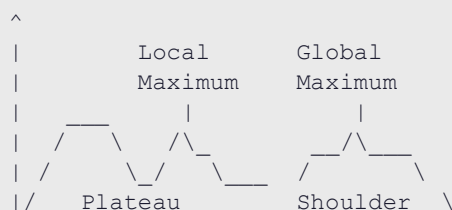
5. Hill Climbing

Hill Climbing is a local search algorithm that continuously moves toward states with a better heuristic value — always going 'uphill' (toward better solutions). It does not maintain a full search tree; it keeps only the current state and moves to the best neighboring state.

WORKED EXAMPLE — TRAVELLING SALESPERSON

Start with a random route visiting all cities. Evaluate its total distance (cost). Swap two cities in the route. If the new route is shorter, accept it. Repeat. The algorithm climbs toward shorter and shorter routes until no single swap improves the solution.

HILL CLIMBING LANDSCAPE (Objective function — higher is better):



+-----> State Space

Problems with Hill Climbing:

- (1) Local Maximum --> Gets stuck at a peak that is not globally best
- (2) Plateau --> Flat region; no clear direction to move
- (3) Ridges --> Diagonal peaks; hard for individual steps

Variants to overcome these problems:

- * Steepest-Ascent --> Evaluates ALL neighbors, picks the best one
- * Random Restart --> Restarts from a random state when stuck
- * Simulated Annealing --> Allows occasional bad moves early on

Fig 3: The hill climbing landscape showing local vs global maxima.

INDUSTRY APPLICATIONS

Machine Learning: Gradient Descent is hill climbing in continuous parameter space — used in every neural network training (TensorFlow, PyTorch).

VLSI Chip Design: Optimizing circuit layouts by making local swaps to minimize wire length.

Job Scheduling: Finding near-optimal task assignment in operating systems and cloud platforms.

Recommendation Systems: Netflix/Spotify fine-tuning recommendation weights using local optimization.

6. A* Algorithm

A* (pronounced 'A-star') is the most famous and widely used heuristic search algorithm. It combines BFS (finds shortest path) with greedy search (uses heuristic). It evaluates each node using the evaluation function:

$$f(n) = g(n) + h(n)$$

where $g(n)$ = actual cost from start to node n , and $h(n)$ = estimated (heuristic) cost from n to goal. A* always expands the node with the lowest $f(n)$ value.

WORKED EXAMPLE — MAP NAVIGATION

Find the shortest route from Arad to Bucharest on a Romanian road map.

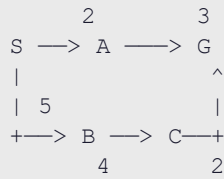
$g(n)$ = actual road distance covered so far.

$h(n)$ = straight-line distance to Bucharest (admissible — actual road distance is always $\geq$ straight-line).

A* combines both values to efficiently find the truly optimal route.

A* Example: Finding path from S to G

GRAPH (numbers = edge costs):



Heuristic values $h(n)$ to goal G:

$h(S)=7, h(A)=3, h(B)=5, h(C)=2, h(G)=0$

Step-by-step execution (Open list sorted by f):

Start: $f(S) = g(S) + h(S) = 0 + 7 = 7$

Expand S:

A: $g=2, h=3 \rightarrow f=5$ <- lowest, expand next

B: $g=5, h=5 \rightarrow f=10$

Expand A:

G: $g=2+3=5, h=0 \rightarrow f=5$ GOAL REACHED!

Optimal Path: $S \rightarrow A \rightarrow G$ (total cost = 5)

(Path $S \rightarrow B \rightarrow C \rightarrow G$ costs $5+4+2 = 11$ -- much worse)

Fig 4: A* uses $f(n)=g(n)+h(n)$ and always finds the globally optimal path.

KEY INSIGHT

A* is guaranteed to find the OPTIMAL solution if the heuristic $h(n)$ is:

- (1) Admissible — never overestimates the true cost to the goal.
- (2) Consistent — $h(n) \leq \text{cost}(n \text{ to } n') + h(n')$ for every successor n' .

It is the gold standard of single-agent pathfinding algorithms.

INDUSTRY APPLICATIONS

Google Maps / Apple Maps / Waze: Real-time route planning uses variants of A* with live traffic as heuristic.

Video Games: Characters in GTA, Age of Empires, and Minecraft use A* to navigate around obstacles in real-time.

Robotics (NASA, Boston Dynamics): Mars rovers and autonomous robots plan collision-free paths using A*.

Airlines (Amadeus, Sabre): Flight routing and seat allocation optimization systems.

7. AO* Algorithm & Problem Reduction

Some problems are best solved by decomposing them into sub-problems. AO* works on AND-OR graphs — a special structure where OR nodes represent choices (one sub-problem must be solved) and AND nodes represent decomposition (ALL sub-problems must be solved).

DEFINITION — AND-OR GRAPH

OR Node: To solve this node, solve any ONE of its children (alternative strategies — just one is needed).

AND Node: To solve this node, solve ALL of its children (all sub-goals must be achieved together).

AO* finds the minimum-cost solution graph in such AND-OR structures.

WORKED EXAMPLE — SOLVING A MATHS EXAM

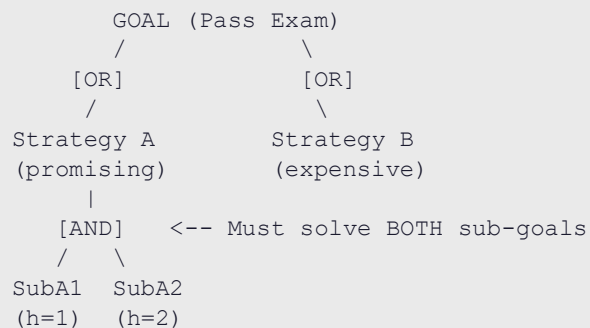
Goal: Pass the exam.

OR decomposition: Study thoroughly OR memorize key topics (alternatives — just one needed).

AND decomposition: Study thoroughly = Understand theory AND Solve practice problems AND Revise (all three are required).

AO* finds the least-cost combination across AND/OR choices.

AO* on AND-OR Graph:



$$f(\text{Strategy A}) = 0 + h(\text{SubA1}) + h(\text{SubA2}) = 0 + 1 + 2 = 3 \quad \text{CHOSEN}$$

$$f(\text{Strategy B}) = 10 \quad (\text{higher cost, not chosen})$$

AO* selects Strategy A, then solves SubA1 and SubA2 independently.

Fig 5: AO* on an AND-OR graph. AND nodes require all children; OR nodes need just one.

INDUSTRY APPLICATIONS

Medical Diagnosis (IBM Expert Systems): Decomposing diagnoses into AND/OR combinations of symptoms.

Automated Theorem Proving: Breaking mathematical proofs into AND-decomposed sub-lemmas.

Project Management (Jira, MS Project): Task decomposition trees where all sub-tasks must complete.

8. Game Playing & Adversarial Search

In classical search, the agent controls everything. In adversarial search, there is an opponent (adversary) who is actively trying to prevent the agent from reaching the goal. This models two-player competitive games like Chess, Tic-Tac-Toe, Checkers, and Go.

FORMAL GAME MODEL

A game is formally defined by six components:

- (1) Initial State — the starting board configuration.
- (2) Players — typically MAX (the AI) and MIN (the opponent).
- (3) Actions — legal moves available at each state.
- (4) Transition Model — the result of applying each action.
- (5) Terminal Test — determines if the game is over (win/loss/draw).
- (6) Utility Function — the final score: e.g., +1 (AI wins), 0 (draw), -1 (opponent wins).

REAL-LIFE ANALOGY

In a chess match, you think: 'If I move my knight here, what is the best move my opponent can respond with? And then what is my best response to that?' This recursive reasoning — thinking ahead about both your moves AND your opponent's best counter-moves — is exactly what adversarial search algorithms do.

Games are difficult because the search space is enormous. Chess has approximately 10^{120} possible games — far more than the number of atoms in the observable universe. Smart algorithms must prune and estimate instead of exhaustively searching every possibility.

9. Mini-Max Algorithm

The Mini-Max algorithm is the foundation of game-playing AI. It assumes both players play optimally. The MAX player (AI) tries to maximize the utility score; the MIN player (opponent) tries to minimize it. The algorithm builds a game tree and backs up values from terminal states to the root.

WORKED EXAMPLE — TIC-TAC-TOE

MAX = X (the AI), MIN = O (the human opponent).

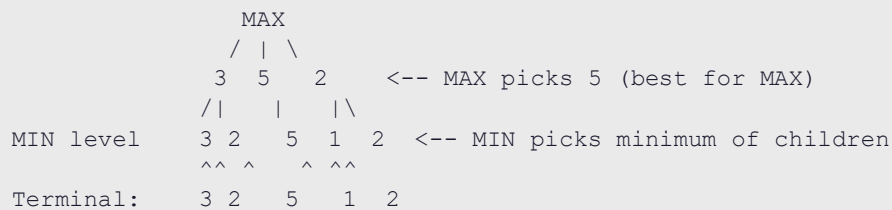
At each terminal state, the utility is: +1 (X wins), 0 (draw), -1 (O wins).

MAX picks the move that leads to the highest backed-up value.

MIN picks the move that leads to the lowest backed-up value.

Both players are assumed to play perfectly (optimally).

MINI-MAX EXAMPLE (simplified game tree):



Mini-Max Procedure:

1. Generate the full game tree down to terminal states.
2. Apply the utility function at all leaf nodes.
3. Back up values: MIN nodes take MINIMUM, MAX nodes take MAXIMUM.
4. Root node chooses the action leading to the backed-up maximum.

Pseudocode:

```

MINIMAX(node, depth, isMaxPlayer):
  if terminal(node): return utility(node)
  if isMaxPlayer:
    best = -INF
    for each child: best = max(best, MINIMAX(child, d-1, false))
    return best
  else:
    best = +INF
    for each child: best = min(best, MINIMAX(child, d-1, true))
    return best

```

Fig 6: Mini-Max tree. MAX nodes pick maximum; MIN nodes pick minimum. Values propagate up.

INDUSTRY APPLICATIONS

Deep Blue (IBM, 1997): Defeated world chess champion Garry Kasparov using mini-max at 12-ply depth, evaluating 200 million positions per second.

Stockfish (Chess Engine): World's strongest open-source chess engine — mini-max core with sophisticated extensions.

Board Game AI: Othello, Checkers, Connect Four, Backgammon — all use mini-max as the fundamental decision engine.

Economic Simulations: Modeling adversarial markets and pricing wars between competing companies.

10. Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of Mini-Max that eliminates branches that cannot possibly influence the final decision. It does not change the result — it reaches the same answer much faster by pruning irrelevant branches of the game tree.

DEFINITION — ALPHA AND BETA VALUES

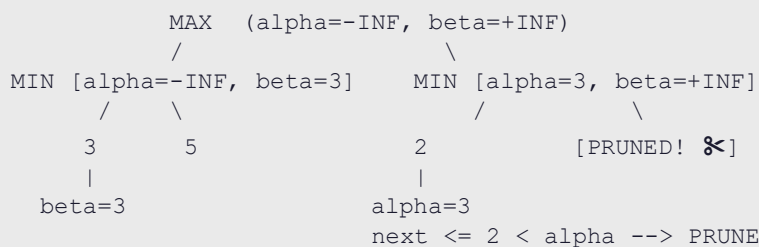
alpha (α): The best value MAX can guarantee so far (starts at $-\infty$).

beta (β): The best value MIN can guarantee so far (starts at $+\infty$).

Pruning condition: If at any node $\alpha \geq \beta$, that entire subtree is pruned.

Neither player would ever choose to go down that branch — so there is no need to explore it.

ALPHA-BETA PRUNING EXAMPLE:



Detailed Trace:

Visit left subtree of MAX:

MIN sees leaf 3 --> $\beta = \min(\text{INF}, 3) = 3$

MIN sees leaf 5 --> $\beta = \min(3, 5) = 3$ (no update)

MIN returns 3 to MAX --> $\alpha = \max(-\text{INF}, 3) = 3$

Visit right subtree of MAX:

MIN sees leaf 2 --> $\beta = \min(\text{INF}, 2) = 2$

Now $\alpha(3) \geq \beta(2)$ --> PRUNE remaining children! ✂

MIN returns 2 to MAX --> $\max(3, 2) = 3$

MAX picks 3 (left branch) -- SAME RESULT as full mini-max, fewer nodes!

Efficiency Gain:

Mini-Max: $O(b^d)$ nodes explored

Alpha-Beta: $O(b^{d/2})$ nodes explored (best case)

For Chess at depth 8: ~17 million nodes --> ~4,000 nodes!

Fig 7: Alpha-Beta pruning. Subtrees are cut once we know they cannot affect the result.

KEY INSIGHT

Alpha-Beta pruning can reduce the effective branching factor to its SQUARE ROOT in the best case.

This means it can search TWICE AS DEEP as plain Mini-Max in the same time.

Move ordering (trying best moves first) dramatically improves pruning effectiveness.

Every serious game-playing engine uses Alpha-Beta pruning.

INDUSTRY APPLICATIONS

Every serious game-playing AI uses Alpha-Beta: Stockfish, Leela Chess Zero, and Komodo all use Alpha-Beta as their core search backbone.

AlphaGo (Google DeepMind, 2016): Combined Alpha-Beta with Monte Carlo Tree Search and Neural Networks to defeat world Go champion Lee Sedol.

Online Games: Chess.com and Lichess use Alpha-Beta engines to generate position analysis and hints for players.

11. Evaluation Functions

Since game trees are too large to expand fully, search is stopped at a certain depth cutoff. At that point, a non-terminal node's value is estimated using an evaluation function (also called a static evaluator). This replaces the utility function for intermediate states.

DEFINITION

An evaluation function $\text{Eval}(s)$ estimates the expected utility of state s from the perspective of the MAX player.

A good evaluation function:

- (1) Returns the true utility at terminal states (win/loss/draw).
- (2) Is fast to compute — called millions of times per move.
- (3) Strongly correlates with the actual probability of winning.

WORKED EXAMPLE — CHESS EVALUATION FUNCTION

Material Count: Pawn=1, Knight=3, Bishop=3, Rook=5, Queen=9, King=infinity.

Positional Factors: Center control bonus, King safety, Pawn structure quality, Piece mobility.

Stockfish computes: $\text{Eval} = \text{Material advantage} + \text{Positional bonuses}$ — evaluated in microseconds.

Modern engines like AlphaZero replace this with a neural network trained on millions of self-play games.

CHESS EVALUATION FUNCTION (simplified):

$$\text{Eval}(\text{state}) = \text{Material} + \text{Position} + \text{Mobility} + \text{King Safety}$$

Material Calculation:

```
White pieces: Knight(3) + Rook(5) + Queen(9) + Bishop(3) = 20 pts
Black pieces: Knight(3) + Rook(5) + Bishop(3)           = 11 pts
Material advantage = +9 (strong advantage for White)
```

Position Bonus (center control):

White pawns on center squares = +0.5 per pawn
 Black pawns on center squares = +0.5 per pawn

Mobility:

White legal moves = 35 --> +0.1 per available move
 Black legal moves = 18 --> fewer moves = worse position

King Safety:

White King castled safely = +1.0
 Black King exposed = -1.5

Final Eval $\approx$ +11.5 (White has a strong, winning advantage)

Modern Neural Network Evaluation (AlphaZero, Leela Chess Zero):

Input: 8x8x112 feature planes (piece positions, move history)

Output: Win probability in [0.0, 1.0]

Learns patterns from millions of self-play games — no hand-crafting

Fig 8: Evaluation function for Chess. Modern engines use neural networks.

Properties of a Good Evaluation Function

- **Accuracy:** Should correlate highly with actual win probability.
- **Speed:** Must be computed in microseconds — called millions of times per move.
- **Terminal Consistency:** Must agree with the actual game outcome at terminal nodes.
- **Granularity:** Should distinguish many levels of advantage, not just win/lose.

INDUSTRY APPLICATIONS

AlphaGo / AlphaZero (DeepMind, Google): Replaced hand-crafted evaluation with a deep neural network trained via self-play — defeating all human champions at Go, Chess, and Shogi simultaneously.

Financial Trading Bots: Evaluation functions for assessing market states in algorithmic trading systems.

Video Game AI (AAA Games): Halo Infinite, StarCraft II — game agents use evaluation functions to assess battlefield positions in real-time.

12. Quick Revision Summary

Algorithm	Type	Complete?	Optimal?	Key Idea
BFS	Uninformed	Yes	Yes (unit cost)	Level by level; uses queue
DFS	Uninformed	No	No	Deepest first; uses stack

Algorithm	Type	Complete?	Optimal?	Key Idea
Hill Climbing	Local Search	No	No	Move to best neighbor; can get stuck
A*	Informed	Yes	Yes (admissible h)	$f(n)=g(n)+h(n)$; expands lowest f first
AO*	AND-OR Search	Yes	Yes	Problem decomposition; AND/OR graphs
Mini-Max	Adversarial	Yes	Yes (vs optimal opponent)	MAX maximizes; MIN minimizes; backup values
Alpha-Beta	Adversarial	Yes	Yes	Prune $\alpha \geq \beta$ branches; $O(b^d/2)$

EXAM TIPS — IMPORTANT POINTS TO REMEMBER

BFS = shortest path guaranteed. DFS = memory efficient but NOT optimal.

A* is BFS + heuristic. Always remember: $f(n) = g(n) + h(n)$.

Hill Climbing's three failure modes: Local Maxima, Plateaus, and Ridges.

Mini-Max: MAX picks MAXIMUM, MIN picks MINIMUM. Values back-propagate from leaves.

Alpha-Beta does NOT change Mini-Max result — it only speeds it up by pruning useless branches.

AO* is for AND-OR graphs where problems naturally decompose into sub-problems.

An admissible heuristic NEVER overestimates the true cost to the goal.